Channel-Aware Backdoor Attacks Against Federated Infostealer Malware Classification Using Dynamic API-Call and Network Representations

1st Mochamad Asryl Aziz
BPS-Statistics Indonesia
Bantaeng Regency
Bantaeng, South Sulawesi, Indonesia
mochamadasryl93@gmail.com

2nd Moh. Jabir Mubarok
Department of Electrical Engineering
Faculty of Intelligent Electrical and Informatics Technology
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
jabirmubarok@gmail.com

3rd Eka Fitria
Department of Informatics
Faculty of Mathematics and Natural Science
Universitas Syiah Kuala
Banda Aceh, Indonesia
ekafitria2@mhs.usk.ac.id

***Abstract*—Federated learning (FL) facilitates collaborative malware model training while preserving the privacy of local data. However, its dependence on updates submitted by participating clients exposes the system to backdoor poisoning attacks. This paper evaluates channel-aware backdoor attacks against federated infostealer malware family classification using dynamic API-call and network representations. Across five Windows infostealer families of 100 samples each, we compare RGB-stack and opacity-blend representations over SmallCNN, MobileNetV2 and ResNet18; RGB-stack is used because it separates the channels explicitly, with ResNet18 its strongest backbone at 78.67% accuracy and 78.84% macro-F1. Under five-client FedAvg the clean baseline reaches 82.67% accuracy and 82.55% macro-F1 over three seeds, and we then inject targeted AgentTesla-to-FormBook triggers into the red/API, green/network, blue/fusion and full-RGB channels. Both the blue/fusion and full-RGB triggers reach 100% attack success rate (ASR) over three seeds while keeping clean performance close to the baseline, and in seven of eight trigger controls the target rate is identical with and without the trigger, so the ASR is not an artefact of the pattern. The novelty is to treat the representation's channels as the attack surface: a trigger confined to the fusion channel alone matches one spanning all three, even though that channel is derived from the other two. Multi-Krum does not mitigate this in a graded way; for each trigger it leaves the backdoor fully effective on one of the three seeds, and on full RGB it costs clean accuracy, 84.00% to 76.00%. Channel-aware backdoors are therefore a serious threat to federated malware classifiers in this controlled IID setting, and motivate representation-aware defenses.**

***Index Terms*—Federated learning, malware classification, backdoor attack, robust aggregation, dynamic malware representation.**

I. Introduction

Infostealer malware targets authentication artifacts such as credentials, browser data and session cookies, which can enable account compromise and unauthorized access [1], [2]. Classifying infostealer families therefore matters for threat intelligence, triage and incident response.

Dynamic analysis captures runtime behavior beyond static file properties: API-call sequences represent host-level execution [3] and network artifacts capture communication patterns [4], and both can be encoded into CNN-compatible images [5].

Federated learning (FL) lets participants train such a classifier together without moving raw data to a central server [6], but its reliance on client-submitted updates lets a malicious client inject backdoor behavior [7].

Most FL backdoor work targets general image or model-level poisoning rather than channel-specific behavior in malware representations [7]. This matters because an RGB-stack representation places API-call, network and fused information in separate channels [5], so a trigger in one channel need not behave like a trigger in another, and colour-space triggers are known to be effective in image-based poisoning [8]. That behavior remains underexplored on dynamic API-call and network representations, and this paper investigates it in federated infostealer malware classification.

The main contributions of this paper are as follows:

- We construct dynamic API-call and network representations from Cuckoo Sandbox reports of five infostealer families.
- We select the main representation–backbone pair by comparing RGB-stack and opacity blend with SmallCNN, MobileNetV2, and ResNet18.
- We evaluate channel-aware backdoor attacks in FL using red/API, green/network, blue/fusion, and full-RGB triggers, with trigger-control and Multi-Krum defense analysis.

II. Related Work

A. Dynamic Malware Representation

Malware classification uses static, dynamic or hybrid representations; dynamic analysis captures runtime behavior such as API calls, file and registry operations, process activity and network traces [9]. The API-call [3], network [4] and RGB-fusion [5] work introduced in Section I underpins the representations used here.

B. Federated Learning for Malware Classification

Federated learning (FL) trains a global model across distributed clients without sharing raw data [10], which suits malware classification because security data is often spread across organizations or sandboxes [6]. It stays vulnerable, however, because the server depends on client-submitted updates and cannot see the local training data [7].

C. Backdoor Attacks and Robust Aggregation

Backdoor attacks allow malicious FL participants to implant hidden target behavior while preserving clean-input performance [11]. In malware classification, this may cause a triggered source family to be misclassified as an attacker-specified target family [7]. Robust aggregation methods such as Krum and Multi-Krum identify reliable client updates by comparing their distances to other updates [12], while coordinate-wise median and trimmed mean limit the influence of extreme update values [13]. However, recent defense work shows that robust aggregation may still be insufficient against stealthy backdoor objectives [14]. Backdoors have also been studied on malware classifiers directly, through explanation-guided feature-space triggers [15] and family-selective triggers [16]. Both target static features of a centralized model; we attack a federated one through the channels of a dynamic-behavior image.

III. Methodology

Figure 1 shows the overall workflow. The subsections below follow its blocks in order: dataset creation, representation construction, model selection, the federated setup, the threat model and channel-aware trigger, and the defenses and trigger control. Section IV then reports the results.

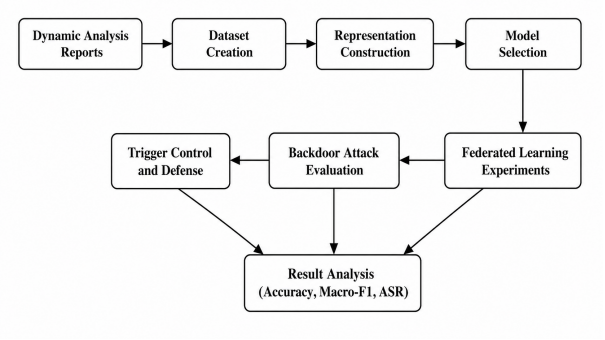

Fig. 1. Overall research workflow of the proposed study.

A. Dataset Creation

The dataset is built from Cuckoo Sandbox reports for five Windows infostealer families: AgentTesla, FormBook, SalatStealer, StealC and Vidar. The sandbox runs each sample in isolation and yields the API-call traces and network artifacts used here [3], [17].

API-call events are converted into a 16×16 category-by-time tile by normalizing the execution timeline, mapping API names into predefined behavior categories, and counting category occurrences over fixed time intervals [18]. Network artifacts are reconstructed from packet captures using flow-level identifiers. To reduce sample-level leakage, only the session with the largest payload is selected for each malware sample, then truncated or padded and reshaped into a 28×28 network tile [4]. Figure 2 illustrates this pipeline, and Table I summarizes the resulting dataset.

Tiles are aligned per sample and recorded in a manifest organized by family, which supports stratified splitting and reproducibility [19]. At training time the 8-bit images are normalized to $[-1, 1]$.

With 100 samples per family the dataset holds 500 representations, split 70%–15%–15% into training, validation and test partitions, stratified by family.

TABLE I
Dataset Summary and Stratified Split

Family	Total	Train	Val.	Test
AgentTesla	100	70	15	15
FormBook	100	70	15	15
SalatStealer	100	70	15	15
StealC	100	70	15	15
Vidar	100	70	15	15
Total	500	350	75	75

B. Dynamic API Call and Network Representations

Two representation strategies turn the dynamic artifacts into CNN-compatible images [19]: RGB-stack and opacity blend.

In RGB-stack the API and network tiles are grayscaled, min–max normalized per image and resized to 128×128; red carries API-call behavior, green network behavior [5], and blue is the edge map of their average, $B = \mathrm{FindEdges}((R+G)/2)$, with $\mathrm{FindEdges}$ a 3×3 edge-detection convolution. Blue is thus a deterministic function of the other two channels, adds no independent information, and is by far the emptiest: 54.5% of its pixels are exactly zero. Opacity blend instead modulates the colour API image by a network-derived texture, so none of its channels corresponds to a single source; RGB-stack is used here because only it separates the sources by channel.

Figure 3 shows representative processed malware representations used in this study.

C. Model Selection

Before the FL and backdoor experiments, we evaluate RGB-stack and opacity blend with three CNN backbones: SmallCNN, MobileNetV2, and ResNet18. SmallCNN is a three-block network (32/64/128 channels, Conv 3×3 + BN + ReLU, max-pooling after the first two blocks, adaptive average pooling, then a linear layer to five classes); MobileNetV2 and ResNet18 are the efficient and residual references [20]–[22], all trained from scratch. The two representations use different split files, so Table III is not a paired comparison across representations: RGB-stack is chosen because channel separation is required here, and macro-F1 only selects the backbone within it.

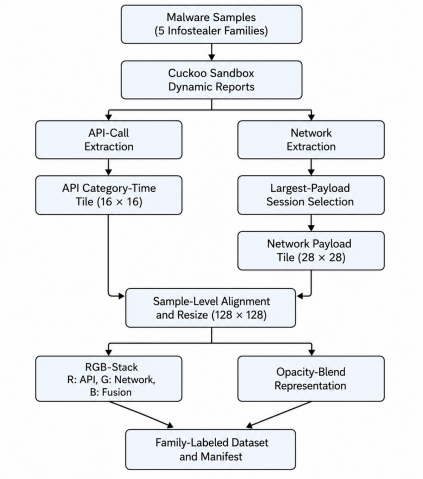

Fig. 2. Dataset creation pipeline from dynamic analysis reports to dynamic API-call and network representations.

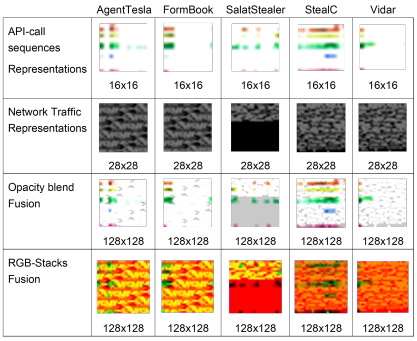

Fig. 3. Representative processed malware representations generated from dynamic API-call and network packet artifacts. Rows illustrate different representation types, while columns correspond to the five infostealer families.

D. Federated Learning Server and Clients

The selected representation–backbone pair is trained in a simulated FL setting with one central server and five clients, shown in Fig. 4. Each client holds 70 images, 14 per family, so the partition is balanced and IID. Clients train locally; the server distributes the global model and aggregates updates without seeing raw data [6].

In each communication round, clients send their updated parameters to the server, which aggregates them with FedAvg:

$$w^{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_k^{t+1}, \tag{1}$$

where w^t is the global model at round t, w_k^{t+1} the model trained locally by client k, n_k the number of samples on client k, n the total, and $K = 5$ the number of clients. The figure writes the model as ω.

In the attack setting, one of the five clients acts as a malicious client that injects poisoned local updates into the FL process [23].

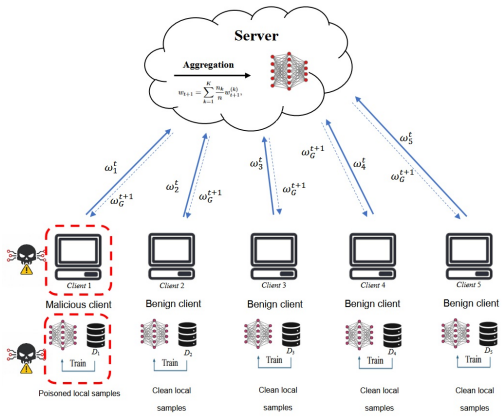

Fig. 4. Federated learning architecture used in this study, consisting of one central server and five clients. One client is malicious in the backdoor attack setting.

E. Threat Model and Channel-Aware Trigger

We assume a targeted backdoor with one malicious client [7]: it inserts a square image-level trigger into AgentTesla training images and relabels them FormBook [8]. AgentTesla is a prominent malware-as-a-service family [24] and the two are behaviorally related, so the pair is a realistic same-category target. The poison rate is 20% of the attacker's 14 AgentTesla training images, i.e. two images per round, resampled every round. The trigger is a white 12×12 square written bottom-right after normalization, so $T(\cdot)$ writes into one channel for red, green and blue and into all three for full RGB. The malicious client is client 0 and participates in every round; its capability is pure data poisoning, with no update scaling and no model replacement, so [11] is background rather than the attack used here.

Because RGB-stack preserves channel-level semantics, four trigger settings are evaluated: red/API, green/network, blue/fusion and full RGB. ASR is the fraction of triggered source-class samples classified as the target class [25]:

$$\mathrm{ASR} = \frac{|\{x_i : y_i = y_s,\ f(T(x_i)) = y_t\}|}{|\{x_i : y_i = y_s\}|}, \tag{2}$$

where y_s is the source label, y_t is the target label, $T(\cdot)$ denotes the trigger insertion function, and $f(\cdot)$ denotes the trained classifier.

F. Defense

Multi-Krum is the primary defense, taken as a standard robust-aggregation baseline: it scores client updates by their distances to other updates and aggregates those with the lowest scores [12]. Let w_k^{t+1} denote the local model update submitted by client k at communication round $t+1$. The pairwise distance between two client updates is computed as:

$$d_{k,j} = \left\| w_k^{t+1} - w_j^{t+1} \right\|_2^2, \tag{3}$$

For each client k, the Multi-Krum score is defined as:

$$s_k = \sum_{j \in \mathcal{N}_k} d_{k,j}, \tag{4}$$

where $\mathcal{N}_k$ denotes the set of closest neighboring updates to client k. Here $K = 5$ and $f = 1$, so the score uses the two nearest updates and the $|\mathcal{S}| = 2$ lowest-scoring of five are averaged. L_2-norm clipping, coordinate-wise median and trimmed mean are screened alongside it [13]. As a control, the same triggers are applied at test time to the clean FL model, which never saw poisoned data, so its target predictions are baseline family confusion.

The server keeps the subset $\mathcal{S}$ of lowest-scoring updates and forms the defended global model. All clients hold 70 images, so the sample-weighted average reduces to:

$$w^{t+1} = \frac{1}{|\mathcal{S}|} \sum_{k \in \mathcal{S}} w_k^{t+1}. \tag{5}$$

G. Experimental Setup

Dynamic analysis and model training run on the separate environments listed in Table II.

TABLE II
EXPERIMENTAL ENVIRONMENT

Component	Specification
Dynamic analysis	Cuckoo Sandbox on Ubuntu
Training environment	Windows, Python / Anaconda
Framework	PyTorch with CUDA 12.8
GPU	NVIDIA GeForce RTX 3080, 12 GB VRAM

H. Training Configuration

All representations are resized to 128×128 pixels. The selected ResNet18 has five output classes and is trained with cross-entropy loss. Training uses a batch size of 16, learning and weight decay rates of 10^{-4}, 50 FL rounds, and two local epochs per round. One deviation must be stated: the seed-42 clean baseline in Table VI and the seed-42 trigger controls come from a 30-round, one-local-epoch run. Retraining it under the common budget gives 0.7867 accuracy, 0.7869 macro-F1 and a control rate of 6/15 rather than 4/15, so the reported run sets a higher bar for the clean-performance claim and a lower one for the trigger-control argument. No ASR result is affected; every backdoor and defense run uses the common budget. ResNet18 is trained from scratch with AdamW, and the reported FL model is the final-round global model, with no best-validation selection; the validation split is not used in the backdoor runs, so the per-round curves in Figs. 5–7 are test-set curves. Clean accuracy, macro-F1, and ASR are reported as primary metrics, and the main experiments are repeated over three seeds: 42, 123, and 2026.

IV. RESULTS AND DISCUSSION

A. Backbone Selection Results

Table III shows the backbone selection results using seed 42. Among the RGB-stack backbones ResNet18 is the strongest, at 78.67% accuracy and 78.84% macro-F1, and is used for all later experiments; the opacity-blend rows come from a different split and are listed for reference only.

TABLE III
BACKBONE SELECTION STUDY USING SEED 42

Representation	Backbone	Acc.	Macro-F1
Opacity blend	SmallCNN	0.5333	0.5298
Opacity blend	MobileNetV2	0.6933	0.6903
Opacity blend	ResNet18	0.7733	0.7730
RGB-stack	SmallCNN	0.5200	0.5144
RGB-stack	MobileNetV2	0.7200	0.7200
RGB-stack	**ResNet18**	**0.7867**	**0.7884**

B. Channel-Aware Backdoor Sweep

Table IV shows that blue/fusion and full-RGB triggers achieve 100% ASR, while red/API and green/network triggers do not, and the blue/fusion trigger reaches perfect ASR without modifying all RGB channels. The difference is categorical. Pooled over the triggered samples, blue/fusion and full RGB differ from their trigger controls at Fisher exact $p = 3.5 \times 10^{-19}$ and $p = 2.6 \times 10^{-18}$, and blue differs from red and green at $p = 4.0 \times 10^{-8}$, whereas red and green are indistinguishable from their own controls (both $p = 1$, 5/15 against 4/15, seed 42 only). Pooling is legitimate because the split is re-drawn per seed. The ordering follows the intensity of the clean images: over all 500 images the mean value inside the trigger region is 241.7 in R, 81.5 in G and 10.2 in B, and 37.7% of red trigger-region pixels are already at least 254, where the trigger changes nothing. Contrast explains red's failure but not green's, whose contrast is 207. Since blue is a deterministic edge map of the other two, its effectiveness cannot come from information they lack; what distinguishes it is that it is almost empty, so the trigger is the only strong response there.

TABLE IV
CHANNEL-AWARE BACKDOOR SWEEP USING SEED 42

Trigger	Clean Acc.	Macro-F1	ASR
Red/API	0.8533	0.8524	0.3333
Green/network	0.8400	0.8419	0.3333
Blue/fusion	0.8400	0.8395	1.0000
Full RGB	0.8400	0.8401	1.0000

C. Defense Screening

Four defenses are screened on a single seed before the multi-seed evaluation. Screening every defense across every seed would multiply the training budget by the number of seeds for candidates that may not be worth carrying forward, and all runs share one GPU, so the screening narrows the candidates cheaply and only the surviving defense is then evaluated across seeds.

Table V gives a seed-42 screening against both strongest attacks: clipping, coordinate-wise median and trimmed mean all leave ASR at 1.0000. Only Multi-Krum reduces it, and only under full RGB (1.0000 to 0.4000, clean accuracy 0.8400 to 0.7600), so Multi-Krum is taken to the multi-seed evaluation.

TABLE V
SINGLE-SEED DEFENSE SCREENING AGAINST STRONGEST ATTACKS USING SEED 42

Trigger	Defense	Clean Acc.	Macro-F1	ASR
Blue/fusion	None	0.8400	0.8395	1.0000
Blue/fusion	Clipping	0.8533	0.8550	1.0000
Blue/fusion	Median	0.8133	0.8120	1.0000
Blue/fusion	Trimmed mean	0.8267	0.8244	1.0000
Blue/fusion	Multi-Krum	0.8267	0.8244	1.0000
Full RGB	None	0.8400	0.8401	1.0000
Full RGB	Clipping	0.8533	0.8550	1.0000
Full RGB	Median	0.8400	0.8388	1.0000
Full RGB	Trimmed mean	0.8133	0.8145	1.0000
Full RGB	Multi-Krum	0.7600	0.7542	0.4000

D. Multi-Seed Results

Table VI shows that blue/fusion and full-RGB backdoors achieve 100% ASR across three seeds while maintaining clean performance comparable to the clean FL baseline.

In the trigger controls the target rate is identical with and without the trigger in 7 of the 8 runs, so what the control measures is the clean model's baseline AgentTesla-to-FormBook confusion rather than any effect of the trigger; the 100% ASR under poisoning cannot be attributed to the trigger pattern.

Figures 5 and 6 show clean macro-F1 holding steady while ASR climbs to 100%.

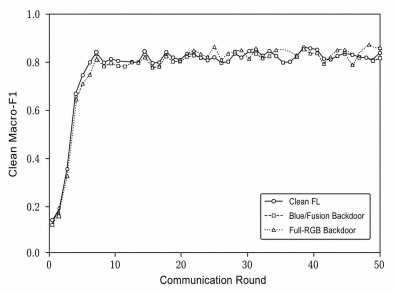

Fig. 5. Clean macro-F1 over FL communication rounds for the clean FL baseline and the strongest backdoor settings, using seed 42.

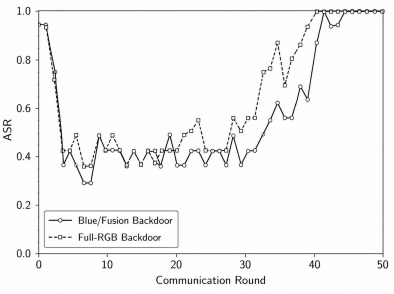

Fig. 6. Attack success rate (ASR) over FL communication rounds for the strongest blue/fusion and full-RGB backdoor settings, using seed 42.

The first rounds are not meaningful: the global model is still close to initialization and collapses onto one or two classes, producing the early spike before clean accuracy rises.

E. Defense Analysis

The last two rows of Table VI report Multi-Krum per seed as well as on average, because the mean is misleading here. For both triggers the across-seed ASR range is at least 0.5, with nothing in between: on some seeds the backdoor remains fully effective, on the others most of it is suppressed. The outcome is bimodal rather than partial, and 0.5111 ± 0.4286 describes no individual run. This also explains the contradiction with the seed-42 screening — precisely the seed at which blue/fusion + Multi-Krum fails. The outcome tracks how often the poisoned update survives selection: across the six runs the malicious client is retained in 8–50% of rounds, and that rate correlates with the final ASR ($r = 0.893$, $p = 0.017$, $n = 6$). With $f = 1$ and $|\mathcal{S}| = 2$ of five, the poisoned update is not an outlier in parameter space, so keeping it is close to a coin flip the attacker need only win often enough — consistent with robust aggregation being insufficient against stealthy objectives [14], [23]. Multi-Krum does reduce ASR against FedAvg ($p = 1.5 \times 10^{-8}$ blue, $p = 9.1 \times 10^{-7}$ full RGB) but costs utility, lowering full-RGB clean accuracy from 0.8400 to 0.7600 on seed 42. No paired clean-performance difference across the three seeds is significant, that drop included (all $p \geq 0.42$, low-powered at $n = 3$).

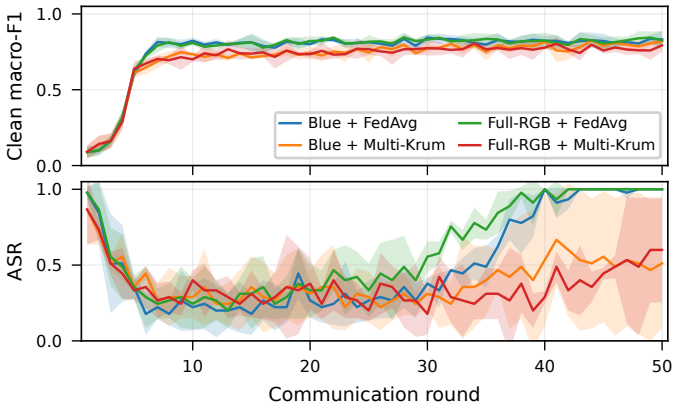

Fig. 7. Per-round clean macro-F1 and ASR across three random seeds for the blue/fusion and full-RGB backdoor settings under FedAvg and Multi-Krum. Shaded areas denote standard deviation across seeds.

Figure 7 gives the per-round view across the three seeds. The backdoor models track the clean baseline in macro-F1, both triggers reach 100% ASR under FedAvg, and under Multi-Krum the ASR curves stay unstable across rounds. Consistent with Table VI, Multi-Krum does not settle at a stable partial suppression level: for each trigger one seed shows no suppression at all, so it is unreliable as a standalone defense.

V. CONCLUSION

This paper evaluated channel-aware backdoor attacks against federated infostealer malware classification using dynamic API-call and network representations. Blue/fusion and

TABLE VI
MAIN MULTI-SEED RESULTS UNDER IID FEDERATED SETTING, WITH PER-SEED VALUES

Scenario	Channel	Clean Acc.	Macro-F1	ASR / Target Rate	Per seed (42, 123, 2026)
Clean FL	–	0.8267 ± 0.0134	0.8255 ± 0.0104	–	–
Trigger control	Blue/fusion	0.8267 ± 0.0134	0.8255 ± 0.0104	0.1333 ± 0.1155	4/15, 1/15, 1/15
Trigger control	Full RGB	0.8267 ± 0.0134	0.8255 ± 0.0104	0.1556 ± 0.1018	4/15, 1/15, 2/15
Backdoor, FedAvg	Blue/fusion	0.8311 ± 0.0539	0.8320 ± 0.0539	1.0000 ± 0.0000	15/15, 15/15, 15/15
Backdoor, FedAvg	Full RGB	0.8267 ± 0.0611	0.8262 ± 0.0621	1.0000 ± 0.0000	15/15, 15/15, 15/15
Backdoor, Multi-Krum	Blue/fusion	0.8178 ± 0.0154	0.8184 ± 0.0135	$0.5111 \pm 0.4286^{\dagger}$	15/15, 3/15, 5/15
Backdoor, Multi-Krum	Full RGB	0.7956 ± 0.0407	0.7931 ± 0.0427	$0.6000 \pm 0.3464^{\dagger}$	6/15, 15/15, 6/15

†Across-seed ASR range ≥ 0.5: the outcome is bimodal, so the mean describes no individual run and the per-seed counts (out of 15 source samples) should be read instead.

full-RGB triggers reached 100% ASR across three seeds while keeping clean performance close to the baseline, and in seven of the eight trigger controls the target rate was identical with and without the trigger, so the effect came from poisoning rather than the trigger pattern. Multi-Krum reduced ASR bimodally rather than partially, leaving the backdoor fully effective (15/15) on one of the three seeds for each trigger. Channel-aware backdoors therefore pose a serious threat to federated malware classifiers and motivate representation-aware defenses.

The main limitations are the IID-only partition, a single source–target pair, and three seeds, with the red and green results resting on seed 42 alone. That the blue trigger works because its channel is nearly empty, rather than because of what that channel encodes, is consistent with the measurements but not established; a non-IID partition, a second source–target pair and a contrast-matched trigger are left to future work.

REFERENCES

[1] K. Thomas, F. Li, A. Zand, J. Barrett, J. Ranieri, L. Invernizzi, Y. Markov, O. Comanescu, V. Eranti, A. Moscicki, D. Margolis, V. Paxson, and E. Bursztein, "Data breaches, phishing, or malware?: Understanding the risks of stolen credentials," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1421–1434.

[2] G. Franken, T. Van Goethem, and W. Joosen, "Who left open the cookie jar? A comprehensive evaluation of third-party cookie policies," in *Proc. 27th USENIX Security Symposium (USENIX Security)*, 2018, pp. 151–168.

[3] P. Wu, M. Gao, F. Sun, X. Wang, and L. Pan, "Multi-perspective API call sequence behavior analysis and fusion for malware classification," *Computers & Security*, vol. 148, p. 104177, 2025.

[4] R. E. Davis, J. Xu, and K. Roy, "Classifying malware traffic using images and deep convolutional neural network," *IEEE Access*, vol. 12, pp. 58031–58038, 2024.

[5] B. Xuan, J. Li, and Y. Song, "BiTCN-TAEfficientNet malware classification approach based on sequence and RGB fusion," *Computers & Security*, vol. 139, p. 103734, 2024.

[6] M. Nobakht, R. Javidan, and A. Pourebrahimi, "SIM-FED: Secure IoT malware detection model with federated learning," *Computers and Electrical Engineering*, vol. 116, p. 109139, 2024.

[7] Q. Ren, Y. Zheng, C. Yang, Y. Li, and J. Ma, "Shadow backdoor attack: Multi-intensity backdoor attack against federated learning," *Computers & Security*, vol. 139, p. 103740, 2024.

[8] W. Jiang, H. Li, G. Xu, and T. Zhang, "Color backdoor: A robust poisoning attack in color space," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 8133–8142.

[9] C. Ma, Z. Li, H. Long, A. Bilal, and X. Liu, "A malware classification method based on directed API call relationships," *PLOS ONE*, vol. 20, no. 3, p. e0299706, 2025.

[10] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[11] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. 23rd Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2020, pp. 2938–2948.

[12] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[13] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018, pp. 5650–5659.

[14] Y. Wang, D.-H. Zhai, and Y. Xia, "RoPE: Defending against backdoor attacks in federated learning systems," *Knowledge-Based Systems*, vol. 293, p. 111660, 2024.

[15] G. Severi, J. Meyer, S. Coull, and A. Oprea, "Explanation-guided backdoor poisoning attacks against malware classifiers," in *Proc. 30th USENIX Security Symp.*, 2021.

[16] L. Yang *et al.*, "Jigsaw puzzle: Selective backdoor attack to subvert malware classifiers," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2023.

[17] A. Pektaş and T. Acarman, "Malware classification based on API calls and behaviour analysis," *IET Information Security*, vol. 12, no. 2, pp. 107–117, 2018, doi: 10.1049/iet-ifs.2017.0430.

[18] T. Chen, H. Zeng, M. Lv, and T. Zhu, "CTIMD: Cyber threat intelligence enhanced malware detection using API call sequences with parameters," *Computers & Security*, vol. 136, p. 103518, 2024.

[19] D. Vasan, M. Alazab, S. Wassan, B. Safaei, and Q. Zheng, "Image-based malware classification using ensemble of CNN architectures (IMCEC)," *Computers & Security*, vol. 92, p. 101748, 2020.

[20] D. Zhang, Y. Song, Q. Xiang, and Y. Wang, "IMCMK-CNN: A lightweight convolutional neural network with multi-scale kernels for image-based malware classification," *Alexandria Engineering Journal*, vol. 111, pp. 203–220, 2025.

[21] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[23] P. Fang and J. Chen, "On the vulnerability of backdoor defenses for federated learning," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 37, no. 10, 2023, pp. 11800–11808.

[24] B. Bhanot, V. Kumar, and S. Sarowa, "Hybrid Analysis of Agent-Tesla: A Prominent Malware as a Service," in *Proc. 2025 3rd International Conference on Device Intelligence, Computing and Communication Technologies (DICCT)*, 2025, pp. 17–22.

[25] T. Liu *et al.*, "Beyond traditional threats: A persistent backdoor attack on federated learning," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 38, no. 19, 2024, pp. 21359–21367.